

QTIP: Trellis와 Incoherence Processing을 이용한 LLM 양자화

QTIP: Quantization with Trellises and Incoherence Processing

Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa (2024)

Weight-only PTQ에서 고차원 trellis coding과 빠른 decode를 함께 설계하는 방법

QTIP의 범위: PTQ와 fine-tuning 경로

항목	원전에서 하는 일	발표에서 구분할 점
기본 대상	학습된 LLM weight의 weight-only PTQ	모델을 처음부터 학습하는 방법이 아니다
1MAD:3INST	계산만으로 trellis value를 decode	논문 실험에서는 fine-tuning 없이 평가
HYB	작은 LUT와 계산을 함께 사용	codebook과 아직 양자화하지 않은 block weight를 fine-tune 가능
activation	weight code의 bit rate와 별도	논문은 일반 activation precision을 주장하지 않는다

QTIP은 PTQ이지만, 모든 결과가 training-free라는 뜻은 아니다.

고차원 VQ의 병목과 TCQ의 출발점

방식	d차원 k-bit 표현	양자화 때	추론 때
Unstructured VQ	2의 kd승 개 codeword	최근접 탐색 비용이 지수적으로 증가	codebook을 cache에 두어야 빠른 lookup 가능
TCQ	state walk로 긴 sequence를 표현	additive distortion이면 Viterbi DP	trellis와 value table의 저장·serial decode가 병목
QTIP	bitshift walk와 재계산 code	TCQ를 rounding framework 안에 사용	graph와 큰 codebook 저장을 피하며 병렬 decode를 노림

QTIP은 고차원 표현 자체보다, 그 표현을 저장하고 decode하는 비용을 함께 다룬다.

양자화가 줄이는 대상: layer proxy error

기호	뜻	해석 경계
W 와 $\hat{W}$	원래 linear-layer weight와 quantized 근사	전체 transformer를 한 번에 비교하지 않는다
x	calibration에서 얻은 layer input activation	모든 실제 입력 분포를 뜻하지 않는다
$H = E\{xx^T\}$	input outer product의 평균인 proxy Hessian	end-to-end training loss가 아니다
$el(\hat{W})$	$E\{ (W-hat - W)x ^2\}$	최종 PPL, accuracy, latency를 직접 최소화하지 않는다

QTIP의 최적화 출발점은 layer reconstruction proxy이며 최종 task metric과는 다르다.

RHT 전처리: Gaussian-like source를 만드는 이유

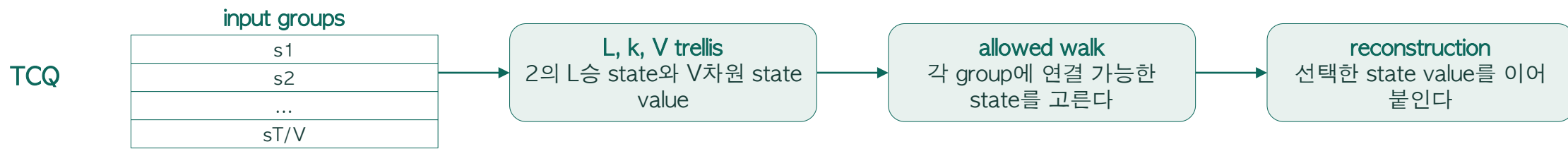
random sign과 Hadamard transform으로 weight와 proxy Hessian의 coordinate를 바꾼다



RHT는 exact Gaussian이 아니라 trellis coding용 근사를 제공한다.

TCQ: codeword 하나가 아니라 허용된 state walk를 고른다

각 state에서 $2^{(kV)}$ 개 edge 중 하나를 선택해 V 개 weight를 표현한다



한 V -weight group의 edge 선택은 kV bits이며, weight당 bit rate는 k 다.

Viterbi가 exact인 범위: additive distortion

- | | | |
|----|---------------|--|
| 01 | group error | 각 s_i 와 state value의 squared error를 계산한다 |
| ↓ | | |
| 02 | DP recurrence | 각 도착 state에 가장 작은 이전 누적 error를 남긴다 |
| ↓ | | |
| 03 | best walk | 마지막 state에서 역추적해 representable walk를 고른다 |
| ↓ | | |
| 04 | boundary | 이 exactness는 group error가 더해질 때만 성립한다 |

Viterbi의 최적 walk와 Eq. 1의 non-additive layer proxy 최소화는 같은 문제가 아니다.

Bitshift trellis: 겹치는 bit window로 병렬 decode

$$j = (i \cdot 2^{(kV)} \bmod 2^L) + c, \quad 0 \leq c < 2^{(kV)}$$



병렬 decode는 일반 TCQ가 아니라 bitshift trellis의 overlapping-window 구조에서 나온다.

교육용 검산: 원전 Fig. 2의 tail-biting walk

L=2, k=1, V=1: 여섯 group의 입력과 재구성 오차를 나누어 본다

group 1-2	group 3-4	group 5-6
targets: 0.54, 0.03	targets: 0.72, 0.19	targets: 0.26, 0.89
states: 00, 01	states: 10, 01	states: 11, 10
recon.: 0.5, 0.1	recon.: 0.8, 0.1	recon.: 0.3, 0.8
errors: 0.0016 + 0.0049	errors: 0.0064 + 0.0081	errors: 0.0016 + 0.0081

각 열은 두 group의 부분값이며, 전체 저장 비트와 총 오차는 다음 표에서 합친다.

Fig. 2: walk 전체의 저장 비트와 오차

부분 group 값과 sequence 전체 결과를 구분한다

항목	전체 값	읽는 의미
state value	00→0.5, 01→0.1, 10→0.8, 11→0.3	Fig. 2의 codebook
walk	00→01→10→01→11→10	인접 state가 1 bit overlap
저장	full 0010110 → tail-biting 001011	마지막 L-kV=1 bit 제거
SSE	여섯 squared error의 합 = 0.0307	여섯 group의 squared error 합

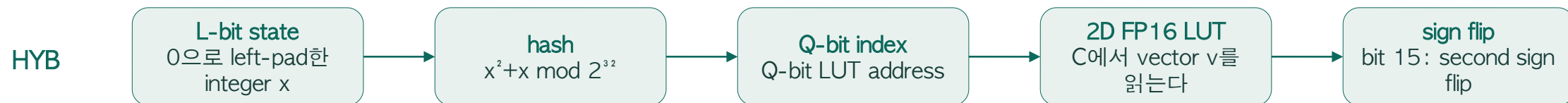
0.0307은 이 walk의 검산값이며, 새롭게 수행한 전체 search의 최적성 증명은 아니다.

세 decoder의 설계: 계산만 또는 작은 lookup

code	state에서 value를 만드는 법	실험·구현 경계
1MAD	LCG word의 네 byte를 합산하고 scale과 shift	lookup-free, NVIDIA GPU에서 최대 4 instructions per weight
3INST	LCG word와 magic FP16 field를 XOR해 두 FP16을 합산	논문은 3 ALU instruction이라고 설명하며 split data movement도 언급
HYB	hash index로 2D FP16 LUT를 읽고 sign bit를 flip	작은 LUT를 fine-tune 가능, 실험 수치는 one-sign-flip version
공통	pseudorandom approximate Gaussian value	true random Gaussian sampling이나 unlimited codebook이 아니다

decoder는 같은 TCQ idea의 구현 선택지이며, cache와 instruction 제약이 서로 다르다.

HYB decoder의 2D vector 복원 경로



L=16, V=2, Q=9의 base LUT 2 KiB와 cache 조건은 구분해야 한다.

Tail-biting: 두 번 Viterbi로 하는 근사

- | | | |
|----|-----------------|---|
| 01 | rotate | S를 $\text{floor}(T/2)$ 만큼 오른쪽으로 회전한다 |
| ↓ | | |
| 02 | first Viterbi | 회전된 sequence를 quantize한다 |
| ↓ | | |
| 03 | extract overlap | start와 end가 공유할 L-kV bit를 읽는다 |
| ↓ | | |
| 04 | second Viterbi | 원 sequence에서 그 overlap를 강제해 다시 quantize한다 |

tail-biting은 bit 낭비를 줄이며, Algorithm 4는 two-pass approximation이다.

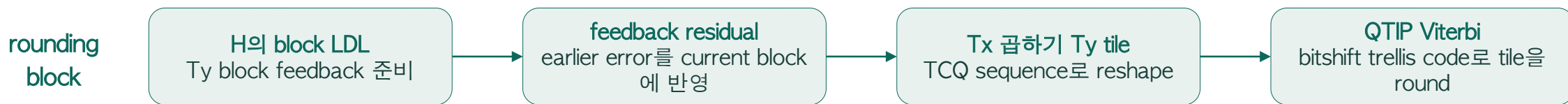
Tail-biting 근사의 Table 2 조건과 수치

k	Algorithm 4 MSE	Optimal MSE	차이
1	0.2803	0.2798	0.0005
2	0.0733	0.0733	0.0000
3	0.0198	0.0198	0.0000
4	0.0055	0.0055	0.0000

이 근접성은 4K개의 T=256 Gaussian sequence, tail-biting (12, k, 1) 결과다.

BlockLDLQ 안에서 TCQ를 쓰는 이유

Hessian-weighted proxy는 additive sequence error가 아니므로 QTIP을 BlockLDLQ의 quantizer Q로 넣는다



BlockLDLQ는 proxy feedback 틀이고, QTIP은 high-dimensional tile codec이다.

Algorithm 5의 block quantization 순서

01	initialize	W-hat를 0으로 두고 H의 Ty-block LDL을 만든다
↓		
02	select block	input dimension에서 다음 Ty column block을 고른다
↓		
03	add feedback	W와 이전 W-hat error, A를 이용해 current x를 만든다
↓		
04	reshape	x를 m/T_x 개의 T_x 곱하기 Ty sequence로 바꾼다
↓		
05	Viterbi and write	각 row sequence를 quantize해 W-hat block에 되돌린다

Viterbi는 Algorithm 5에서 feedback이 반영된 tile에 row-wise로 적용된다.

숫자를 읽기 전 고정해야 하는 실험 조건

결과 묶음	주요 설정	비교할 때의 경계
Table 3	Llama 2, ctx 4096, 1MAD·3INST, L=16, V=1, Tx=Ty=16, no FT	QTIP은 no-FT, QuIP#·AQLM 열린 FT 조건
Table 5-6	HYB, L=16, V=2, Q=9, Tx=Ty=16	Llama 1과 Llama 2는 context와 baseline overhead가 다름
Table 4	RTX 6000 Ada, batch size 1, matrix fusion decoding	throughput은 end-to-end serving latency 일반값이 아님
Appendix	Hessian은 RedPajama calibration sequence에서 생성	model family와 sequence length가 서로 다름

같은 ‘2-bit QTIP’이라도 code, FT, context, hardware가 다르면 같은 실험 행이 아니다.

Table 1 : i.i.d. Gaussian source-MSE 비교

quantizer	차원	MSE	이 표가 말하는 범위
Lloyd-Max scalar	1	0.118	2-bit i.i.d. Gaussian source
QulP# E8P	8	0.089	같은 source-MSE 조건의 VQ baseline
QTIP 1MAD·3INST	256	0.069	L=16 1D TCQ columns
QTIP HYB	256	0.071	L=16 2D TCQ column
distortion-rate lower bound	infinite	0.063	k-bit quantizer가 도달 가능한 distortion의 하한

Table 1은 source-MSE 비교이며 LLM PPL이나 throughput proof가 아니다.

Table 3의 예: Llama 2 Wikitext2 perplexity

model	1MAD no FT	3INST no FT	QuIP# FT	AQLM FT
2-7B, about 3-bit	5.38	5.40	5.41	5.38
2-13B, about 3-bit	4.74	4.74	4.78	4.78
2-70B, about 3-bit	3.27	3.27	3.35	3.36
condition	ctx 4096, L=16, V=1	ctx 4096, L=16, V=1	same table, FT	same table, FT

이 비교는 Table 3의 model, bit-rate, context, fine-tuning labels를 함께 읽어야 한다.

Table 4: RTX 6000 Ada batch- 1 decoding throughput

method	bits	Llama 2 7B tok/s	Llama 2 70B tok/s
FP16	16	55.9	OOM
AQLM	2	81.5	8.78
QuIP#	2	186	22.2
QTIP	2	188	23.5
QTIP	3 / 4	161 / 140	19.1 / 16.3

이 throughput은 RTX 6000 Ada, matrix fusion, batch size 1 의 측정값이다.

Appendix Table 17: GPU가 바뀌면 speed도 바뀐다

GPU	model	2-bit tok/s	3-bit tok/s	4-bit tok/s
RTX 3090	2-7	127	119	109
RTX A6000 Ampere	2-7	116	106	95
RTX 6000 Ada	2-7	188	161	140
RTX A6000 Ampere	2-70	15.0	13.1	11.7
RTX 6000 Ada	2-70	23.5	19.1	16.3

GPU별 speed 측정을 untested hardware의 deployment claim으로 바꾸면 안 된다.

QTIP이 지지하는 결론과 남는 경계

원전이 지지하는 범위	원전만으로 말할 수 없는 범위
bitshift와 codes는 storage·serial decode를 겨냥	모든 hardware에서 항상 더 빠름
additive Viterbi와 Table 2의 close MSE	proxy global optimum·tail-biting exactness
named condition의 PPL·accuracy·throughput	최신 SOTA·universal serving latency
checklist: formal theorem 없음, error bars 없음	작은 표 차이의 statistical superiority

QTIP의 핵심은 RHT, bitshift TCQ, decoder, BlockLDLQ를 조건과 함께 연결하는 데 있다.